

ACTIVITY - I

1. Cross Browser Testing

Cross Browser Testing means checking whether a website or web application works correctly in **different browsers**.

Different browsers may display the same website differently. So testers verify that the application behaves the same everywhere.

Common Browsers

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari
- Opera

What testers check

- Page layout
- Fonts and colors
- Buttons and links
- Images
- Forms
- JavaScript functionality

Example

A login page may look perfect in Chrome but the **login button may not work in Firefox**. Cross browser testing helps find these issues.

Why it is important

- Users use different browsers
- Ensures consistent user experience

- Prevents UI and functionality issues

2. Adhoc Testing

Adhoc Testing is an **informal testing method** where testers test the application **without any test cases or documentation**.

The tester randomly checks the application to find defects.

This is "informal" or "impromptu" testing. There are no written test cases or specific plans.

- **The Approach:** You act like a random user. You click around, try things out of order, and see if you can break something.
- **The Goal:** To find bugs quickly that might have been missed by structured testing. It relies heavily on the tester's intuition and experience.

Characteristics

- No test plan
- No test cases
- Based on tester experience
- Done quickly

Example

A tester opens the application and:

- clicks random buttons
- enters random data
- navigates different pages

If something crashes or behaves wrongly, a bug is reported.

Why it is useful

- Finds hidden defects
- Saves time
- Good for quick testing

Types of Testing:

1. Buddy Testing

Buddy Testing happens when **two people work together** to test the application.

Usually:

- **One is a developer**
- **One is a tester**

How it works

- The developer explains the feature.
- The tester tests the feature immediately.
- Both discuss issues together.

Example

A developer creates a **new login feature**.
The tester sits with the developer and tests it directly.

Advantages

- Quick bug detection
- Better understanding of the feature
- Good collaboration between developer and tester

2. Pair Testing

Pair Testing is when **two testers work together on the same feature**.

Roles

Usually:

- **One tester operates the system**
- **The other tester observes and suggests ideas**

They switch roles during testing.

Example

Two testers test a **shopping cart feature**:

- Tester 1 adds items to cart.
- Tester 2 suggests scenarios like removing items or changing quantity.

Advantages

- More ideas for testing
- Better test coverage
- Faster bug discovery

3. Monkey Testing

Monkey Testing means testing the application with **random inputs and actions**.

The tester does random operations without following any specific steps.

Example

Tester may:

- click buttons randomly
- enter random text
- refresh pages repeatedly
- open many tabs

Goal

To check if the application **crashes or breaks**.

Advantages

- Finds unexpected bugs
- Tests system stability

4. Freestyle Testing

Freestyle Testing means testers test a **specific feature freely without predefined test cases**.

The tester focuses on one module but explores different scenarios.

Example

Testing **login page**:

- enter wrong password
- leave fields empty
- enter special characters
- try multiple login attempts

Advantage

Helps find **hidden defects in a specific module**.

Exploratory Testing

While it sounds like Ad-hoc testing, it's a bit more professional. It's a "think-on-your-feet" approach where you learn, design tests, and execute them all at the same time.

- **The Approach:** You follow a specific "charter" (like "Explore the checkout process"), but you don't have a step-by-step script. You use what you learn from one click to decide where to click next.
- **The Goal:** To discover deep, hidden issues that strict scripts are too "blind" to see.

Exploratory Testing means **learning the application and testing it at the same time**.

The tester explores the system, understands features, and tests them simultaneously.

Key idea

Explore + Learn + Test

Steps

1. Understand the feature
2. Try different scenarios
3. Observe system behavior
4. Report bugs

Example

While testing an **e-commerce website**:

- add products to cart
- remove items
- change quantity
- apply coupons

During exploration, the tester may find unexpected bugs.

UI Validations (User Interface Validation)

UI Validation means checking whether the **user interface elements appear correctly on the screen**.

UI testing ensures the application is **visually correct and user friendly**.

This checks the **visual elements** of the application. It's all about the "look and feel."

- **What you check:**
 - Are the fonts and colors correct?
 - Are the buttons aligned properly?
 - Is the text readable and not overlapping?
- **The Goal:** To ensure the app is aesthetically pleasing and follows the design mockups.

Things checked in UI validation

- Button alignment

- Text spelling
- Fonts and colors
- Image display
- Page layout
- Labels and headings
- Responsive design

Example

On a login page:

- Username field should be visible
- Password field should hide characters
- Login button should be clickable
- Error message should appear properly

Why UI validation is important

- Improves user experience
- Ensures professional design
- Prevents layout issues

Broken Links Testing

This is a simple but vital check to ensure every link on your site actually goes somewhere.

- **What you check:** You click every hyperlink, image link, and menu item.
- **The Result:** If you click a link and get a **404 Error (Page Not Found)**, that is a "broken link."
- **The Goal:** To prevent user frustration and keep your SEO rankings healthy.

Broken Link Testing checks whether all links on a website work correctly.

A **broken link** is a link that does not open the intended page.

Types of broken links

1. **Internal links** – links within the same website
2. **External links** – links to other websites

What testers check

- Clicking link opens correct page
- No **404 error**
- No **page not found**
- No **server error**

Example

If a link "Contact Us" leads to 404 Page Not Found, then it is a broken link.

Why it is important

- Improves user experience
- Helps SEO
- Keeps website reliable

Form Validation Testing

Form Validation Testing checks whether input fields in a form accept **correct data and reject incorrect data**.

Forms are used for:

- Login
- Registration
- Payment
- Contact forms

Types of validations

1. Mandatory Field Validation

Required fields must not be empty.

Example

Username field cannot be blank.

2. Data Format Validation

Checks correct format.

Example:

- Email → test@gmail.com
- Phone number → 10 digits

3. Length Validation

Checks minimum and maximum characters.

Example:

- Password must be **8–16 characters**

4. Numeric Validation

Fields that accept only numbers.

Example:

- Age
- Phone number

5. Error Message Validation

Correct error message should appear.

Example:

- "Email is required"
- "Invalid phone number"

Example Form Testing Scenario

Registration form fields:

- Name
- Email
- Password
- Phone number

Test cases:

- Leave fields empty
- Enter invalid email
- Enter short password
- Enter letters in phone number